laundriroom

Set-up guide: Google Maps, WhatsApp and email

Step-by-step instructions for connecting the three outside services laundriroom uses. Each one is optional: without it the app still works, it just does less.

1. **Google Maps**: delivery fees by distance, and “find the nearest laundry”.
2. **WhatsApp**: order, support and vendor alerts on WhatsApp.
3. **Email (Resend)**: order, account and support emails, and Admin → Emails.

For the laundriroom admin team · Keep keys private: never paste them into chats, emails or documents.

1 Google Maps

What it switches on: delivery fees worked out from the road distance between the laundry branch and the customer, and a **Find the nearest laundry** box on the customer's order screen (from their saved address or their phone's location). Without a key, deliveries use the flat fee and "nearest" uses straight-line distance.

Before you start

- [ ] A Google account you'll keep for the business (not a personal one that might be closed).
- [ ] A bank card for Google Cloud billing. Google gives a free monthly allowance; small volumes usually stay within it.
- [ ] Access to the website's hosting settings (for example Vercel), where the app's settings live.

A. Create the key in Google Cloud

1. Go to **console.cloud.google.com** and sign in. At the top, open the project picker → **New project**. Name it `laundriroom` → **Create**, then select it.
2. Turn on billing: menu ☰ → **Billing** → link a billing account (add the card if asked).
3. Set a spending alert so there are no surprises: **Billing → Budgets & alerts → Create budget**, e.g. ₦20,000 / month, alerts at 50%, 90% and 100%.
4. Turn on the two services the app uses: menu ☰ → **APIs & Services → Library**. Search for and **Enable**:
   - **Geocoding API**: turns addresses into map points.
   - **Routes API**: road distances. (Don't use "Distance Matrix API": it's the old version and new projects can't turn it on.)
5. Create the key: **APIs & Services → Credentials → Create credentials → API key**. Copy it (it starts with `AIza`).
6. Lock the key down: click the key's name → under **API restrictions** choose **Restrict key** and tick only **Geocoding API** and **Routes API** → **Save**. Leave **Application restrictions** on **None**: only the app's server uses the key, never the browser.

B. Give the key to the website

1. Open the hosting dashboard. On Vercel: your project → **Settings → Environment Variables**.
2. Add a variable named exactly `GOOGLE_MAPS_API_KEY`, paste the key as the value, tick **Production** (and Preview if you test there) → **Save**.

> Don't add `NEXT_PUBLIC_` to the name. That prefix would publish the key to every visitor's browser.

3. **Redeploy** so the site picks it up: **Deployments** → latest → ⋯ → **Redeploy**.

C. Switch it on in laundriroom

1. Sign in as the admin → **Admin → Fees & charges → Delivery pricing** → choose **By distance**. If a yellow note says “Google Maps isn’t connected”, the key isn’t in place yet (check step B and the redeploy).
2. Set the **amount per km**, a **base fare** and a **minimum fee** → **Save**. Fees are rounded up to the next ₦50.
3. Put each branch on the map: **Vendor → Branches → Edit → Save** (no changes needed) for every branch. The address is looked up when it’s saved. Do the same for new branches as they’re added.

D. Check it works

- [ ] As a customer, open **Order** → **Use my location**: laundries are listed nearest first with “km away” and the note “distance by road”.
- [ ] Choose **Deliver to me** with an address: the delivery fee shows the distance and the price for it.

> **If something goes wrong,** orders are never blocked: any failed look-up falls back to the flat delivery fee (and straight-line distance for “nearest”). “Couldn’t place that address on the map” usually means the address is too vague. Add the street and area.

Cost control: each “nearest laundry” search asks Google for up to 10 road distances in one request, and each delivery quote asks for one. Keep the budget alert from step A3.

2 WhatsApp

What it switches on: alerts on WhatsApp alongside email (new orders for laundries and admins, order confirmations and support replies for customers, vendor quality notices). It uses Meta's official **WhatsApp Cloud API**. Messages come from a business number you set up, not from someone's personal WhatsApp.

Before you start

- [ ] A **Meta Business account** (business.facebook.com) for laundriroom, with an admin you control.
- [ ] A phone number for laundriroom alerts that is **not** already registered on the WhatsApp or WhatsApp Business app (or one you're ready to move off the app). It must be able to receive an SMS or call for the code.
- [ ] A payment method for Meta: messages the business starts are charged per conversation.

A. Create the WhatsApp app

1. Go to **developers.facebook.com** → **My Apps** → **Create app**. Use case: **Other** → type **Business** → name it `laundriroom` → choose your Business account → **Create**.
2. On the app's page, find **WhatsApp** → **Set up**. Pick the same Business account.
3. **WhatsApp** → **API Setup** → **Add phone number**: enter the display name (`laundriroom`), category, then the number; confirm it with the code Meta sends. The display name is reviewed by Meta, usually within a day or two.
4. Still on **API Setup**, select your number in the "From" list and copy the **Phone number ID** (a long number, *not* the phone number itself).
5. Verify the business (**Business settings** → **Security centre** → **Start verification**). It isn't needed to start, but it raises how many people you can message per day.
6. Add a payment method: **WhatsApp Manager** → **Overview** → **Add payment method**.

B. Make a permanent access token

The token on the API Setup page expires after 24 hours, so make one that doesn't:

1. **business.facebook.com** → **Business settings** → **Users** → **System users** → **Add**. Name it `laundriroom-server`, role **Admin**.
2. With that system user selected → **Assign assets**: add the **laundriroom app** (Full control) and your **WhatsApp account** (Full control).
3. **Generate new token** → choose the laundriroom app → expiry **Never** → tick `whatsapp_business_messaging` and `whatsapp_business_management` → **Generate**. Copy the token now: it's shown only once.

C. Create the message template

WhatsApp only lets a business start a conversation with an **approved template**. laundriroom uses one template for every alert and fills in the message:

1. **WhatsApp Manager → Message templates → Create template**.
2. Category **Utility**, name `laundriroom_alert` (lower case, underscores), language **English**.
3. Body, with exactly one variable: `laundriroom: {{1}}`. For the sample value, write e.g. *Your order ORD-261109-0001 is ready for collection.*
4. **Submit**. Approval usually takes minutes to a few hours. Note the language code shown on the template (`en` or `en_US`): it must match exactly in step D.

D. Connect it in laundriroom

1. **Admin → Integrations → WhatsApp → Connect**.
2. Paste the **Access token** (from B3), the **Phone number ID** (A4), the template name `laundriroom_alert` and its **language code** (C4).
3. **Save & send test**. The app checks the token can see that number and shows its name; any problem shows Meta's own error message. The token is stored encrypted in Supabase Vault and never shown again.
4. Choose which events go out on WhatsApp: **Admin → Communications**. Tick WhatsApp on the events you want.

If you see…	Do this
Error 190 / "token expired"	Make a permanent system-user token (section B) and paste it again.
132001 "template name does not exist"	The template isn't approved yet, or the name/language code differs from WhatsApp Manager.
131030 "recipient not in allowed list"	You're still on Meta's test number. Add and use your own number (A3).
Customer gets nothing	Their account needs a phone number. Nigerian numbers in any format (0803…, +234 803…) are fine.

3 Email (Resend)

What it switches on: order, account and support emails, and broadcasts from Admin → Emails. Password-reset and invite emails are sent by Supabase itself and don't need this.

A. Set up Resend

1. Sign up at **resend.com** with the business email.
2. **Domains → Add domain**: enter the domain you send from (e.g. `laundriroom.ng`). Resend lists 3–4 DNS records (TXT/MX/CNAME).
3. Add those records at your domain provider's DNS settings, then click **Verify** in Resend. It can take from minutes to a few hours.

   > Until the domain is verified, Resend only delivers to the email you signed up with. This is the most common reason test emails fail.

4. **API Keys → Create API key**: name `laundriroom`, permission **Sending access**, domain: yours → **Add**. Copy the key (starts with `re_`).

B. Connect it in laundriroom

1. **Admin → Integrations → Resend (email) → Connect** (or **Edit / test again**).
2. Paste the **API key**, and set the **From address** on your verified domain, e.g. `laundriroom <hello@laundriroom.ng>`.
3. **Save & send test**: a test email goes to your admin email. If it fails, Resend's reason is shown on the card with the fix (wrong key, domain not verified, etc.).

> **Already pasted a key before?** Older versions of the app didn't store the key from this screen, which is why sending said "unauthorized". Paste the key once more: it's now kept encrypted in Supabase Vault.

Final checklist

- [ ] Google Maps: key added to hosting as `GOOGLE_MAPS_API_KEY`, site redeployed, Delivery pricing set to By distance, branches re-saved.
- [ ] WhatsApp: own number added, permanent token, template approved, connected and tested on Integrations, events ticked on Communications.
- [ ] Email: domain verified, key connected and tested on Integrations.
- [ ] Budget alerts set in Google Cloud and a payment method on Meta.

Screens and menu names at Google, Meta and Resend change from time to time. If a button has moved, search the provider's help for the step's title.